



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Practica 1

Investigación de Bases de Datos y Caso de Estudio

Materia: Circuitos Eléctricos

Profesor: Gabriel Hurtado Avilés

Alumnos:

- Damas Garnica Adrian
-
-

Grupo: 3CV4

Resumen

En este documento se presenta la documentación inicial del desarrollo de una base de datos para el desarrollo de dicha base de datos focalizada en un sistema de comandas.

Índice

1	Introducción	1
1.1	Planteamiento del problema	1
1.2	Justificación	1
1.3	Objetivo general	2
1.4	Objetivos específicos	2
1.5	Estado del arte	2
1.6	Productos esperados	2
2	Marco teórico	4
2.1	Dato, información y bases de datos	4
2.1.1	¿Qué es un dato?	4
2.1.2	Información	4
2.1.3	Base de datos	4
2.2	Características de una base de datos	5
2.2.1	Integración	5
2.2.2	Persistencia	5
2.2.3	Redundancia controlada	5
2.2.4	Integridad	6
2.2.5	Independencia de datos	6
2.2.6	Seguridad	6
2.2.7	Concurrencia	7
2.2.8	Recuperación	7
2.3	Archivos contra bases de datos	7
2.3.1	Sistemas de archivos	7
2.3.2	Problemas del enfoque en sistemas de archivos tradicionales	8
2.4	Usuarios de una base de datos	8
2.4.1	Administrador de la base de datos	8
2.4.2	Diseñador de la base de datos	9
2.4.3	Programador de aplicaciones	9
2.4.4	Usuario final	10
2.5	Ciclo de vida de una base de datos	10
2.6	Sistema gestor de bases de datos	11
2.6.1	Módulos de componentes principales	11
2.6.2	Lenguajes del SGBD	12
2.6.3	Clasificación de los SGBD	12
2.7	Sistema de bases de datos	13
2.7.1	Arquitectura de tres niveles (ANSI/SPARC)	13
2.7.2	Independencia de datos	13
2.7.3	Arquitectura centralizada	13
2.7.4	Arquitectura cliente-servidor	14
2.8	Modelos de datos	14
2.8.1	Clasificación por nivel de abstracción	14

3	Caso de Estudio	15
4	Conclusiones	16

1 Introducción

1.1 Planteamiento del problema

Los salones de eventos que ofrecen servicio de banquetes suelen gestionar la toma y el seguimiento de comandas –los pedidos de alimentos y bebidas realizados por cada mesa o por cada invitado durante un evento– de forma manual, apoyándose en comandas de papel, hojas de cálculo o aplicaciones genéricas que no fueron diseñadas para este giro de negocio. Esta forma de operar genera problemas concretos: la información de los pedidos se duplica o se pierde entre el área de meseros, cocina y caja; no existe un registro histórico confiable de qué se sirvió en cada evento, a qué cliente y a qué costo; el control de existencias de insumos se lleva de manera aproximada, lo que provoca faltantes o compras innecesarias; y la elaboración de la cuenta final de cada evento es lenta y propensa a errores porque depende de conciliar manualmente varias comandas en papel.

Este proyecto retoma y da continuidad a un desarrollo realizado en un semestre anterior, en el que se propuso un primer sistema de comandas de uso general. Sin embargo, aquella versión inicial se apoyaba en un sistema enteramente en C y este únicamente trabajaba con archivos en formato ASCII para almacenar la información, esto causa que dicho sistema tenga relaciones mal definidas, escasa normalización y ausencia de reglas de integridad, lo que dificultaba su mantenimiento y su crecimiento. Por ello, el problema que atiende la presente práctica no es únicamente digitalizar la toma de comandas, sino rediseñar, con una metodología formal de modelado entidad-relación, la base de datos que sustenta dicho sistema, de modo que sea capaz de representar correctamente eventos, clientes, mesas, menús, platillos, insumos, meseros y pagos, garantizando consistencia, integridad y escalabilidad de la información, de esta forma concretar con un sistema profesional y funcional para nuestro proyecto final de la materia de bases de datos.

1.2 Justificación

El diseño de una base de datos robusta para un sistema de comandas resulta pertinente porque este tipo de sistema constituye el núcleo operativo de un salón de eventos: de él dependen la asignación de mesas, el control de los pedidos por evento, la disponibilidad de insumos y, en última instancia, la facturación al cliente. Un modelo de datos mal estructurado no solo genera inconsistencias en la información, sino que limita la capacidad del sistema para escalar a medida que el negocio crece o para adaptarse a nuevos requerimientos, como el manejo de menús personalizados, distintos tipos de evento o reportes administrativos.

Desde el punto de vista académico, esta práctica es relevante porque permite aplicar de forma integral los conceptos revisados a la fecha en la materia de Bases de Datos –definición de una base de datos, implementación de una base de datos, modelos entidad relación, uso de docker y uso de PostgreSQL– a un caso de estudio real y ya conocido por el equipo, lo que facilita identificar con claridad los errores de diseño cometidos previamente y corregirlos me-

diante un modelo entidad-relación formal, con entidades, atributos, relaciones y cardinalidades correctamente definidas.

Desde el punto de vista práctico, contar con una base de datos bien diseñada beneficia directamente la operación de un salón de eventos: reduce el tiempo de atención al reducir errores en la toma de comandas, permite conocer con precisión el costo y las existencias de insumos por evento, y facilita la generación de reportes confiables para la toma de decisiones administrativas, sentando además una base sólida sobre la cual continuar el desarrollo del sistema en etapas posteriores del proyecto.

1.3 Objetivo general

Diseñar el modelo entidad-relación de una base de datos para un sistema de comandas de un salón de eventos, que permita gestionar de manera íntegra y consistente la información de clientes, eventos, mesas, menús, platillos, insumos, meseros y pagos, con el fin de sustentar una versión mejorada del sistema desarrollado previamente.

1.4 Objetivos específicos

1. Investigar y comparar la definición formal de dato, información y base de datos, así como las características fundamentales de un sistema gestor de bases de datos.
2. Identificar y documentar, mediante una entrevista con el cliente, los requerimientos de información y las funciones que debe cumplir el sistema de comandas del salón de eventos.
3. Analizar las deficiencias de diseño presentes en la base de datos del proyecto realizado en el semestre anterior, con el fin de evitarlas en la nueva propuesta.
4. Construir el diagrama entidad-relación del sistema de comandas, incluyendo entidades, atributos, relaciones y cardinalidades.
5. Justificar las decisiones de diseño del modelo propuesto, explicando cómo cada entidad y relación contribuye a resolver la problemática planteada.

1.5 Estado del arte

1.6 Productos esperados

Como resultado de esta práctica se espera obtener un conjunto de entregables documentales y técnicos que evidencien tanto el proceso de investigación como el diseño de la base de datos del sistema de comandas.

Los productos esperados son:

- Descripción del caso de estudio, incluyendo la entrevista con el cliente y la lista de requerimientos del sistema de comandas.
- Diagrama entidad-relación del sistema de comandas para el salón de eventos, con entidades, atributos, relaciones y cardinalidades.
- Las bases de la implementación de la base de datos para el proyecto final.

2 Marco teórico

2.1 Dato, información y bases de datos

2.1.1 ¿Qué es un dato?

De acuerdo con Elmasri y Navathe (2016), un dato «es un hecho que es reconocido y que puede ser registrado con un significado implícito» (p. 4). Un ejemplo de dato podrían ser: un nombre, números de teléfono, ubicaciones, direcciones de correo electrónico, entre muchos más.

Por otro lado, Bellinger et al. (2004) indican que los datos «no tienen un significado aparente más allá de su propia existencia», y estos pueden ser o no utilizables para distintos fines (p. 1).

Por ambas partes los autores nos dan a entender que los datos son trozos de información que se pueden utilizar con distintos fines; Bellinger y sus colaboradores nos dicen que más allá de su existencia no tienen otro significado.

2.1.2 Información

Elmasri y Navathe (2016) señalan que una base de datos se constituye por medio de una serie de datos relacionados entre sí de manera lógica y coherente; por lo tanto, la base de datos genera información con los datos que almacena. De esta forma, no es posible construir una base de datos con una variedad aleatoria de datos sin una conexión entre sí (p. 5).

Bellinger et al. (2004) mencionan que «la información son datos a los que se les ha dado significado a través de una conexión relacional» (p. 1).

Ambos autores nos dicen que los datos tienden a tener una conexión racional y lógica para generar información y que este mismo principio es fundamental y constituye a las bases de datos. Un ejemplo de esto podría ser una calle: «calle 1» por sí sola no tiene sentido, pero si se le relaciona con una persona se puede decir que esa persona vive en la «calle 1».

2.1.3 Base de datos

Para Date (2001), una base de datos «es un sistema computarizado que se utiliza para llevar el registro electrónico de información» (p. 2). Los usuarios y administradores pueden realizar acciones enfocadas en la información que almacenan; por ejemplo, pueden agregar más información, modificar la información existente, recuperarla o eliminar información y datos.

Silberschatz et al. (2002) definen diferencias entre un sistema de procesamiento de datos y una base de datos, indicando que un sistema gestor de bases de datos coordina sistemas físicos y

accesos lógicos, y este mismo garantiza el correcto uso de la información en aplicaciones por medio de consultas (p. 4).

El autor Date nos da una versión más figurada y sencilla de lo que es una base de datos, mientras que Silberschatz y sus colaboradores ofrecen una visión más técnica y detallada sobre las diferencias principales que tiene una base de datos frente a un sistema de procesamiento de datos.

2.2 Características de una base de datos

2.2.1 Integración

Elmasri y Navathe (2016) explican que una base de datos funciona como un único repositorio donde se unifican las estructuras de los datos y la información; de esta forma, reemplaza a los archivos independientes e inconexos (p. 10).

Según Silberschatz et al. (2002), la integración es «la consolidación y la centralización de la información dispersa o descentralizada en múltiples archivos», de forma que permite el acceso unificado a la información (p. 3).

Ambos autores nos dicen la misma idea, donde los archivos independientes o descentralizados se conectan de forma conexas en la base de datos.

2.2.2 Persistencia

De acuerdo con Elmasri y Navathe (2016), los datos almacenados tienen la propiedad de persistir de forma duradera en los medios de almacenamiento no volátiles, sobreviviendo a la finalización o ejecución de los programas que los crearon (p. 20).

Date (2001) afirma que, de forma precisa, los datos de la base de datos «persisten» debido a que una vez son aceptados por el SGBD solo pueden ser removidos por una petición al SGBD, pero no como un efecto lateral si algún programa termina su ejecución (pp. 9–10).

En ambas citas los autores nos plantean ideas similares: que dentro de un SGBD los datos y la información persisten, y esta persistencia siempre estará presente sin importar las ejecuciones que haga la aplicación donde se aloja el SGBD, y solo cambiará por medio de peticiones directas al SGBD.

2.2.3 Redundancia controlada

Elmasri y Navathe (2016) señalan que, aunque la norma es eliminar la duplicación mediante la normalización, cuando es necesario duplicar datos de forma intencional para acelerar las

consultas, el SGBD aplica una redundancia controlada para asegurar que el cambio se haga correctamente por igual entre sus copias (pp. 17–18).

Silberschatz et al. (2002) indican que consiste en evitar la duplicación innecesaria de información (propia de los sistemas de archivos tradicionales) para prevenir inconsistencias; si existe redundancia por diseño, el SGBD mantiene la sincronización (p. 4).

Los autores Elmasri y Navathe profundizan más en el aspecto de cómo funciona dicha duplicidad, y en la otra cita se profundiza en cómo se mantiene una sincronización dentro del SGBD.

2.2.4 Integridad

Elmasri y Navathe (2016) mencionan que la integridad garantiza que los datos cumplan con las restricciones definidas en el catálogo del sistema (como la clave primaria), las cuales el SGBD valida automáticamente en cada actualización (pp. 19–20).

Por su parte, la integridad «asegura la corrección y consistencia de los datos mediante el cumplimiento estricto de reglas de validación como claves primarias especificadas por el administrador» (Silberschatz et al., 2002, p. 5).

En este caso los autores dan la misma idea, donde cierta información específica (describen la clave primaria como ejemplo) tiene que cumplir con la característica de integridad.

2.2.5 Independencia de datos

Según Elmasri y Navathe (2016), es la separación entre los programas de aplicación y los datos gracias a la abstracción. Incluye la independencia física (modificar el almacenamiento sin alterar el esquema conceptual) y la independencia lógica (alterar el esquema conceptual sin modificar las aplicaciones externas) (pp. 12, 36–38).

Silberschatz et al. (2002) la definen como la capacidad de alterar el esquema físico o lógico sin requerir la reescritura ni modificación de los programas de aplicación que acceden a la base de datos (p. 11).

En la primera cita se profundiza más sobre cuáles son las partes del sistema que no interfieren con los datos, ya sea de forma física y/o lógica; la segunda cita es más clara y directa.

2.2.6 Seguridad

Consiste en la protección del sistema contra accesos no autorizados o malintencionados mediante controles de acceso discrecionales (GRANT/REVOKE), obligatorios (por niveles), cifrado de datos y defensa contra amenazas como la inyección SQL (Elmasri & Navathe, 2016, p. 21).

Silberschatz et al. (2002) la describen como los mecanismos del SGBD que restringen el acceso a la información únicamente a los usuarios autorizados, otorgando privilegios específicos de lectura, modificación o ejecución (p. 5).

Ambos autores nos hablan de cómo el SGBD tiene diferentes mecanismos de seguridad para con el usuario; en la primera se profundiza más en los controles de acceso.

2.2.7 Concurrencia

Elmasri y Navathe (2016) indican que la concurrencia permite que múltiples usuarios y transacciones modifiquen la base de datos simultáneamente. El subsistema de control de concurrencia aplica bloqueos o marcas de tiempo para garantizar la atomicidad y el aislamiento de las operaciones (pp. 13–14, 21).

Silberschatz et al. (2002) señalan que es la supervisión del acceso simultáneo a los datos que evita anomalías o inconsistencias (como lecturas sucias o actualizaciones perdidas), logrando que las ejecuciones concurrentes equivalgan a ejecuciones seriales (p. 5).

En la segunda cita se describen las consecuencias concretas que pueden pasar en caso de una falta de supervisión del sistema. En la primera cita se describe mejor la característica.

2.2.8 Recuperación

Elmasri y Navathe (2016) la definen como la capacidad del SGBD para restaurar la base de datos a un estado consistente tras un fallo de hardware, software o de sistema, utilizando registros en bitácora (*log*), operaciones UNDO/REDO e hitos de control (p. 21).

Silberschatz et al. (2002) explican que es un conjunto de algoritmos que garantizan las propiedades de atomicidad y durabilidad, deshaciendo los cambios de transacciones no terminadas y asegurando la persistencia de las transacciones confirmadas ante fallos (p. 5).

Ambas citas describen muy bien la característica, pero en la primera se describen las operaciones concretas para ponerlas en práctica.

2.3 Archivos contra bases de datos

2.3.1 Sistemas de archivos

Sistemas de archivos tradicionales: como señalan Elmasri y Navathe (2016, p. 10) y Silberschatz et al. (2002, p. 3), cada aplicación gestiona sus propios archivos de datos de forma independiente, incrustando la estructura de almacenamiento dentro del código fuente de los programas de aplicación.

Sistemas de bases de datos: según Elmasri y Navathe (2016), los datos se almacenan en un repositorio centralizado e integrado. La definición y estructura de los datos no reside en las aplicaciones, sino en un **catálogo o diccionario de datos (metadatos)** administrado por el SGBD (pp. 11–12).

2.3.2 Problemas del enfoque en sistemas de archivos tradicionales

1. **Redundancia de datos:** Elmasri y Navathe (2016, p. 17) y Silberschatz et al. (2002, p. 4) indican que la falta de planificación centralizada ocasiona que la misma información o datos se almacenen repetidamente, y eso ocasiona desperdicio de espacio en unidades físicas.
2. **Inconsistencia de datos (datos duplicados y contradictorios):** Silberschatz et al. (2002) mencionan que al modificar un dato en un archivo pero omitir la actualización en otro, el sistema almacena información contradictoria para la misma entidad, destruyendo la confiabilidad del sistema y volviendo a caer en la redundancia de datos (p. 4).
3. **Dependencia programa-datos:** de acuerdo con Elmasri y Navathe (2016, p. 11) y Silberschatz et al. (2002, p. 4), si la estructura física y el formato de las declaraciones de archivos están codificados dentro de cada programa, cualquier cambio en la estructura de un archivo exige modificar, recompilar y probar todos los programas que acceden a él.
4. **Dificultad de acceso:** Silberschatz et al. (2002) explican que los sistemas de archivos solo permiten consultas predeterminadas mediante programas compilados. No existen lenguajes de consulta como el lenguaje SQL, por lo que obtener un reporte o siquiera alguna consulta requiere codificar un nuevo programa desde cero (p. 4).

2.4 Usuarios de una base de datos

En un entorno de bases de datos coexisten una variedad de usuarios y perfiles técnicos y operativos involucrados en diferentes áreas de uso en las bases de datos.

Según Elmasri y Navathe (2016), en los sistemas corporativos «muchas personas participan en el diseño, uso y mantenimiento de una base de datos grande con cientos o miles de usuarios», clasificándolos primordialmente como los «actores en el escenario» (p. 15).

2.4.1 Administrador de la base de datos

Responsabilidades: el administrador de la base de datos tiene la máxima autoridad técnica y operativa en el entorno. Según Elmasri y Navathe (2016), «en un entorno de base de datos, el recurso primario es la propia base de datos y el recurso secundario es el DBMS y el soft-

ware asociado; administrar estos recursos es responsabilidad del administrador de la base de datos» (p. 15). Date (2001) también nos dice que el «administrador de datos es un profesional IT. El trabajo del DBA consiste en crear la base de datos real e implementar los controles técnicos necesarios para hacer cumplir las diversas decisiones de las políticas hechas por el administrador de datos» (p. 16).

Permisos: el DBA tiene el nivel más alto de autorización dentro del entorno (privilegios de administrador). Como explican Elmasri y Navathe (2016), «la mayoría de los sistemas de base de datos contienen comandos privilegiados que solo pueden ser utilizados por el personal del DBA» (p. 15).

2.4.2 Diseñador de la base de datos

Responsabilidades: el diseñador se encarga de determinar el modelo lógico de la implementación del sistema y el uso de la información de este mismo. En palabras de Elmasri y Navathe (2016), «los diseñadores de bases de datos son responsables de identificar los datos que se almacenarán en la base de datos y de elegir las estructuras apropiadas para representar y almacenar estos datos», y detallan que los diseñadores deben «comunicarse con todos los posibles usuarios de la base de datos para comprender sus requisitos y crear un diseño que satisfaga estas necesidades» (p. 15).

Permisos: los permisos que tienen se centran en el modelado estructural del entorno, desarrollo y *testing*; posee permisos para modificar la estructura lógica de las bases de datos, tablas e índices.

2.4.3 Programador de aplicaciones

Responsabilidades: este rol tiene la responsabilidad de hacer funcionar el SGBD dentro de una aplicación por medio de código y herramientas de desarrollo. Según exponen Elmasri y Navathe (2016), «los analistas de sistemas determinan los requisitos de los usuarios finales y desarrollan especificaciones para transacciones programadas estándar, mientras que los programadores de aplicaciones implementan estas especificaciones como programas; luego prueban, depuran, documentan y mantienen estas transacciones enlatadas» (p. 16).

Permisos: los desarrolladores tienen permisos de manipulación de datos sobre los esquemas de desarrollo y *testing*. Elmasri y Navathe (2016) señalan que su labor exige interactuar con precompiladores que extraen comandos DML incrustados en lenguajes anfitriones, por lo que disponen de privilegios para ejecutar operaciones como SELECT, INSERT, UPDATE y DELETE, además de compilar procedimientos almacenados y disparadores (*triggers*) (p. 43).

2.4.4 Usuario final

Responsabilidades: los usuarios finales son los destinatarios directos de los SGBD, aquellos a los que va dirigido el uso del sistema. Elmasri y Navathe (2016) dicen que «los usuarios finales son las personas cuyos trabajos requieren acceso a la base de datos para consultar, actualizar y generar informes; la base de datos existe principalmente para su uso» (p. 15).

Permisos: los permisos para los usuarios finales se rigen por el principio de mínimo privilegio, donde solo tienen permitidos ciertos permisos según sea el caso. Según Elmasri y Navathe (2016), «a los usuarios o grupos de usuarios se les asignan números de cuenta protegidos por contraseñas. Algunos usuarios solo pueden tener permiso para recuperar datos, mientras que a otros se les permite recuperar y actualizar» (p. 19).

2.5 Ciclo de vida de una base de datos

Como explican Elmasri y Navathe (2016), una base de datos real «posee un ciclo de vida de muchos años, por lo que el DBMS debe ser capaz de mantener el sistema de base de datos permitiendo que evolucione conforme los requerimientos cambian a lo largo del tiempo» (p. 8). Esto se realiza con el fin de evitar que el proyecto fracase o genere inconsistencias. Chen (1976) plantea que el desarrollo debe dividirse en etapas progresivas que van desde la idea en el negocio hasta la puesta en marcha técnica, produciendo cada una de ellas un resultado hasta llegar al final (pp. 9–11).

Planificación del proyecto: antes de modelar, el equipo evalúa si el proyecto tiene sentido práctico y financiero. Se identifican los problemas actuales (como archivos duplicados o procesos manuales lentos), se define el alcance, los costos previstos y si la infraestructura actual soportará la solución. El entregable sería un documento de estudio de viabilidad y plan del proyecto.

Recolección y análisis de requerimientos: esta etapa consiste en entender qué necesita la organización o proyecto. De acuerdo con Elmasri y Navathe (2016), «el diseño de una base de datos completamente nueva comienza con una fase denominada especificación y análisis de requerimientos. Estos requisitos se documentan en detalle para que puedan mantenerse, modificarse y transformarse fácilmente en una implementación» (p. 8). El documento entregable para esta etapa sería un reporte de requerimientos específicos.

Diseño conceptual: una vez que se tienen los requerimientos se crea un modelo semántico que describe el diseño conceptual de la base de datos. Chen (1976) fundamentó este paso demostrando que representar el mundo real mediante entidades y relaciones permite organizar la información de forma natural y sin sesgos tecnológicos (pp. 9–10). El entregable para esta etapa es un esquema conceptual donde se hagan diagramas entidad-relación.

Diseño lógico: Elmasri y Navathe (2016) explican que esta etapa es el puente entre el modelo conceptual y el paradigma del gestor elegido, usualmente el relacional (p. 62). Siguiendo las reglas de derivación introducidas por Chen (1976, p. 28) y formalizadas por Elmasri y Navathe

(2016, p. 289), el entregable es el esquema lógico relacional donde se presentan tablas con las definiciones de llaves e integridad referencial.

Diseño físico: de acuerdo con Date (2001, p. 43) y Elmasri y Navathe (2016, p. 37), es la adaptación del diseño lógico al almacenamiento real en disco o memoria secundaria. Como indican Elmasri y Navathe (2016), el objetivo es «estructurar los datos en el almacenamiento garantizando un buen rendimiento en las consultas y transacciones más frecuentes» (p. 643). El entregable es un esquema físico e interno donde se emplea código SQL y sentencias DDL con parámetros definidos.

Implementación: Date (2001, p. 41) y Elmasri y Navathe (2016, p. 31) señalan que en esta etapa el diseño cobra vida en el motor, compilando el DDL para crear las tablas vacías en el DBMS y pasando al estado inicial cargando los datos reales o migrados. Asimismo, Elmasri y Navathe (2016) explican que a la par se enlazan las aplicaciones cliente (mediante *drivers* como JDBC u ODBC) y se somete el sistema a pruebas de concurrencia y seguridad (pp. 43–45). El entregable es la base de datos en producción junto con sus bitácoras operativas y reportes.

Operación: Date (2001, p. 42) y Elmasri y Navathe (2016, pp. 15, 45) describen esta fase como la vida cotidiana del sistema en producción. Los usuarios finales operan las aplicaciones y el administrador (DBA) supervisa que todo marche con fluidez: aplica respaldos periódicos, ajusta índices si las consultas se vuelven lentas y modifica estructuras cuando el negocio crece o cambia. El entregable en muchos casos son los informes de mantenimiento de la base de datos, entre otros.

2.6 Sistema gestor de bases de datos

Un Sistema Gestor de Bases de Datos (DBMS) es el conjunto de software que media entre los archivos físicos en disco, los usuarios y las aplicaciones clientes. Como señalan Elmasri y Navathe (2016), «el DBMS es un sistema computarizado que facilita los procesos de definir, construir, manipular y compartir bases de datos entre diversos usuarios y programas de aplicación» (p. 6).

2.6.1 Módulos de componentes principales

De acuerdo con la arquitectura descrita por Elmasri y Navathe (2016, pp. 41–44) y Silberschatz et al. (2002, pp. 4–5), el motor se organiza en el procesador de consultas y el gestor de almacenamiento, los cuales se conforman por los siguientes módulos:

Compilador de DDL: interpreta las sentencias de definición de esquemas y vuelca las definiciones y restricciones semánticas directamente en el catálogo del sistema en forma de metadatos.

Compilador de DML y precompilador: el precompilador extrae las instrucciones DML incrus-

tadas en código anfitrión, mientras que el compilador DML las traduce a llamadas de bajo nivel comprensibles por el procesador de ejecución.

Optimizador de consultas: examina las distintas formas de resolver una consulta declarativa y elige el plan de ejecución con menor costo estimado de CPU y operaciones de entrada/salida (E/S). Según explican Elmasri y Navathe (2016), «la optimización de consultas es la actividad realizada por un optimizador de consultas en un DBMS para seleccionar la mejor estrategia disponible para ejecutar la consulta» (p. 691).

Procesador de base de datos en tiempo de ejecución: es el motor central que recibe las consultas compiladas y las transacciones parametrizadas, verifica los permisos y esquemas con el catálogo y comanda la ejecución de las operaciones.

2.6.2 Lenguajes del SGBD

En los sistemas actuales basados en SQL, las instrucciones conviven en una interfaz integrada, pero conceptualmente corresponden a cuatro sublenguajes con objetivos distintos (Elmasri & Navathe, 2016, pp. 38–41; Silberschatz et al., 2002, pp. 42–44, 74).

Lenguaje de Definición de Datos (DDL): utilizado por los diseñadores y el DBA para especificar el esquema conceptual, crear tablas, índices, dominios y reglas de integridad estructural (Elmasri & Navathe, 2016, p. 39).

Lenguaje de Manipulación de Datos (DML): permite a los programas y usuarios interactivos buscar, insertar, modificar o borrar registros. Como puntualizan Elmasri y Navathe (2016), «los DML de alto nivel, como SQL, pueden especificar y recuperar muchos registros en una sola sentencia; por lo tanto, se denominan orientados a conjuntos (*set-oriented*) y declarativos» (p. 40).

Lenguaje de Control de Datos (DCL): proporciona las primitivas de seguridad para gestionar los derechos de acceso de las cuentas de usuario sobre los objetos de la base de datos (Elmasri & Navathe, 2016, p. 19; Silberschatz et al., 2002, p. 74).

Lenguaje de Control de Transacciones (TCL): delimita las unidades atómicas de trabajo garantizando que las modificaciones se apliquen de forma permanente o se descarten por completo si ocurre un error (Silberschatz et al., 2002, pp. 175–176).

2.6.3 Clasificación de los SGBD

Relacionales (RDBMS / SQL): organizan la información en tablas o relaciones bidimensionales regidas por llaves y álgebra relacional, como PostgreSQL, MySQL, Oracle, etc. Chen (1976) demostró que estos modelos derivan limpiamente de las entidades y relaciones del mundo real (pp. 9–11).

NoSQL y almacenamiento Big Data: diseñados para escalabilidad horizontal masiva y es-

quemados flexibles o semiestructurados (Elmasri & Navathe, 2016, pp. 51–52).

2.7 Sistema de bases de datos

Para evitar que los programas de aplicación dependan directamente de la forma en que los datos se almacenan físicamente en el disco, los sistemas de bases de datos modernos separan las aplicaciones de la estructura física subyacente.

2.7.1 Arquitectura de tres niveles (ANSI/SPARC)

Propuesta por el comité ANSI/SPARC, esta arquitectura busca aislar las aplicaciones de usuario de los detalles físicos del almacenamiento (Elmasri & Navathe, 2016, p. 36). Como señalan Elmasri y Navathe (2016), «el objetivo de la arquitectura de tres esquemas es separar las aplicaciones de usuario de la base de datos física» (p. 37).

Nivel interno: posee un esquema interno que describe la estructura de almacenamiento físico de la base de datos (archivos, métodos de acceso, índices, compresión y ordenamiento de registros) (Elmasri & Navathe, 2016, p. 37).

Nivel conceptual: describe la estructura lógica global de toda la base de datos para la comunidad de usuarios, ocultando los detalles de almacenamiento (Elmasri & Navathe, 2016, p. 37).

Nivel externo: comprende diversos esquemas externos o vistas de usuario. Cada vista describe únicamente la porción de datos de interés para un grupo específico de usuarios, ocultando el resto de la base de datos por razones de simplicidad o seguridad (Elmasri & Navathe, 2016, p. 37).

2.7.2 Independencia de datos

La independencia de datos es la capacidad de modificar el esquema en un nivel del sistema sin tener que alterar el esquema del nivel superior ni los programas de aplicación (Elmasri & Navathe, 2016, pp. 37–38; Silberschatz et al., 2002, p. 4).

2.7.3 Arquitectura centralizada

En este esquema, toda la funcionalidad del sistema se ejecuta en una sola máquina: el procesamiento del SGBD, la ejecución de los programas de aplicación y la lógica de la interfaz de usuario (Elmasri & Navathe, 2016, pp. 46–47). Los usuarios se conectaban originalmente mediante terminales sin capacidad de cómputo que solo mostraban texto e interfaces de pantalla (Elmasri & Navathe, 2016, p. 46).

2.7.4 Arquitectura cliente-servidor

Aprovecha la capacidad de cómputo en el extremo del usuario (PC, estaciones de trabajo o dispositivos móviles), dividiendo las cargas de trabajo entre dos o más capas lógicas (Elmasri & Navathe, 2016, p. 47).

2.8 Modelos de datos

Un modelo de datos es una colección integrada de conceptos destinados a describir la estructura de una base de datos, incluyendo los tipos de datos, sus relaciones, restricciones semánticas y el conjunto de operaciones básicas de manipulación y consulta. Como señalan Elmasri y Navathe (2016), «una característica fundamental del enfoque de bases de datos es que proporciona cierto nivel de abstracción de datos. Un modelo de datos, como una colección de conceptos que pueden utilizarse para describir la estructura de una base de datos, proporciona los medios necesarios para lograr esta abstracción» (p. 32).

2.8.1 Clasificación por nivel de abstracción

De acuerdo con Elmasri y Navathe (2016, pp. 33–34), los modelos se ordenan en tres categorías según la cercanía que guardan con la percepción humana frente a los dispositivos de almacenamiento:

Modelos de datos conceptuales: emplean conceptos intuitivos muy cercanos a la percepción de los usuarios finales y alejados de los detalles de hardware (Elmasri & Navathe, 2016, p. 33). Sus elementos fundamentales son las entidades (objetos del mundo real), los atributos (propiedades descriptivas) y las relaciones (asociaciones entre entidades) (Chen, 1976, p. 11; Elmasri & Navathe, 2016, p. 33). El ejemplo canónico es el modelo Entidad-Relación y su extensión EER (Elmasri & Navathe, 2016, p. 33).

Modelos de datos de representación o implementación: se sitúan en un punto intermedio: son comprensibles para los usuarios finales, pero se corresponden directamente con las estructuras organizativas que implementan los motores comerciales. Ocultan los detalles de almacenamiento en sectores de disco, pero representan los datos mediante estructuras de registros o tablas (Elmasri & Navathe, 2016, p. 33).

Modelos de datos físicos: describen cómo se almacenan físicamente los datos en los medios secundarios (discos magnéticos o unidades de estado sólido) (Elmasri & Navathe, 2016, p. 33). Como especifican Elmasri y Navathe (2016), «los modelos de datos físicos describen cómo se almacenan los datos como archivos en la computadora representando información como formatos de registro, ordenamiento de registros y rutas de acceso» (p. 34).

3 Caso de Estudio

4 Conclusiones

Referencias

- Bellinger, G., Castro, D., & Mills, A. (2004). *Data, information, knowledge, and wisdom*. Universidade Federal de Minas Gerais (UFMG). Consultado el 18 de septiembre de 2026, desde <https://homepages.dcc.ufmg.br/~amendes/SistemasInformacaoTP/TextosBasicos/Data-Information-Knowledge.pdf>
- Chen, P. P.-S. (1976). The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9-36. <https://doi.org/10.1145/320434.320440>
- Date, C. J. (2001). *Introducción a los sistemas de bases de datos* (S. L. M. Ruíz Faudón, Trad.; 7.^a ed.). Pearson Educación.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.^a ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2002). *Database system concepts: Instructor's manual* (4.^a ed.). McGraw-Hill Higher Education.